Arithmetic with Python

COSC 1210 Exam Prep: A Visual Execution Trace

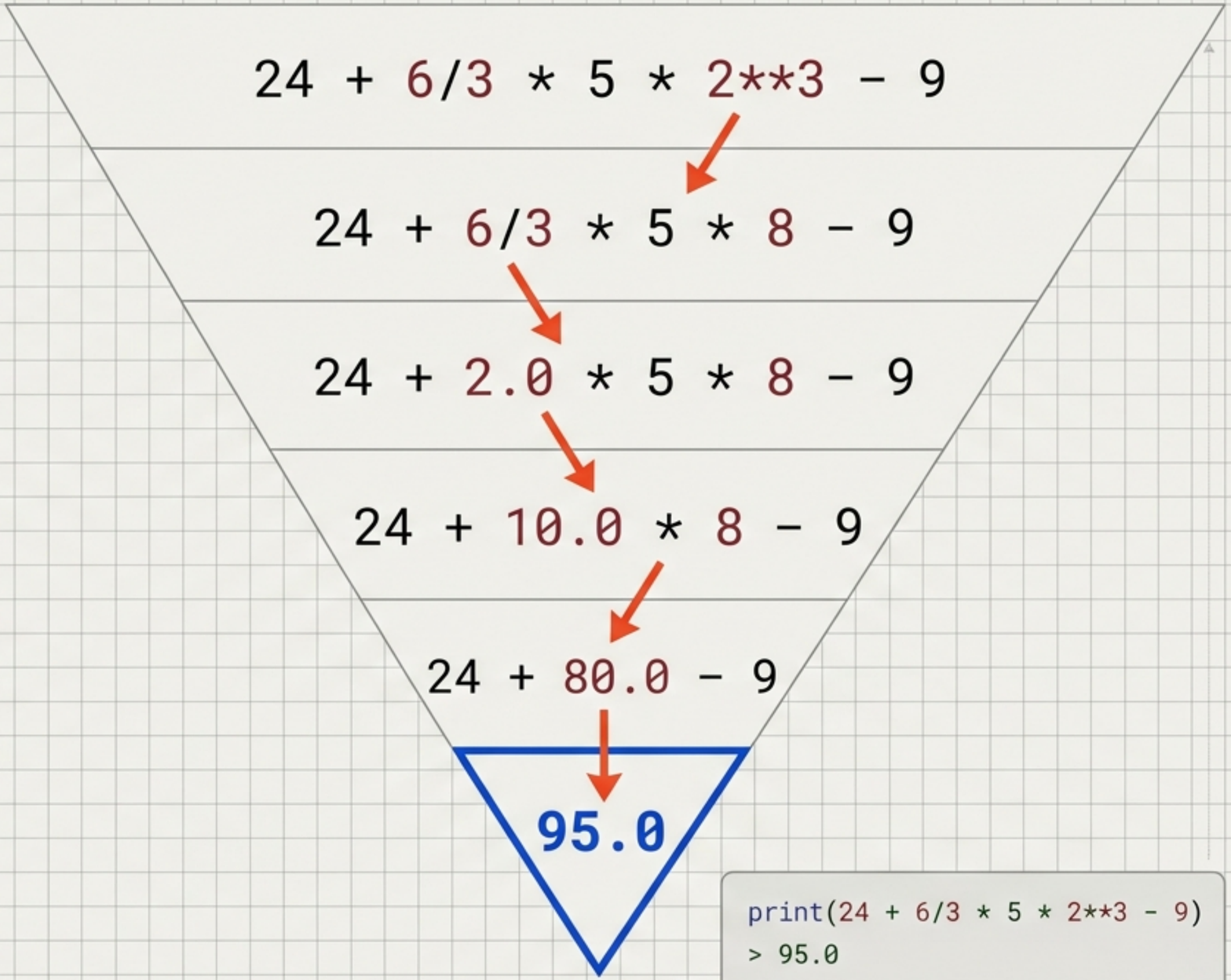
Focus	Expressions, Math Functions, Types, and Error Debugging
Status	Pre-quiz Reference Guide
Design	Optimized for self-paced review and recall

Expressions & Operator Precedence (PEMDAS)

Rule: An expression is any combination of values and operators. Python evaluates

evaluates left-to-right using operator precedence (PEMDAS).

Core Principle: The machine is compelled to evaluate and reduce an expression to a single value.




```
print(24 + 6/3 * 5 * 2**3 - 9)
> 95.0
```

Concept Panel: Division (/) and Floating-Point Illusions

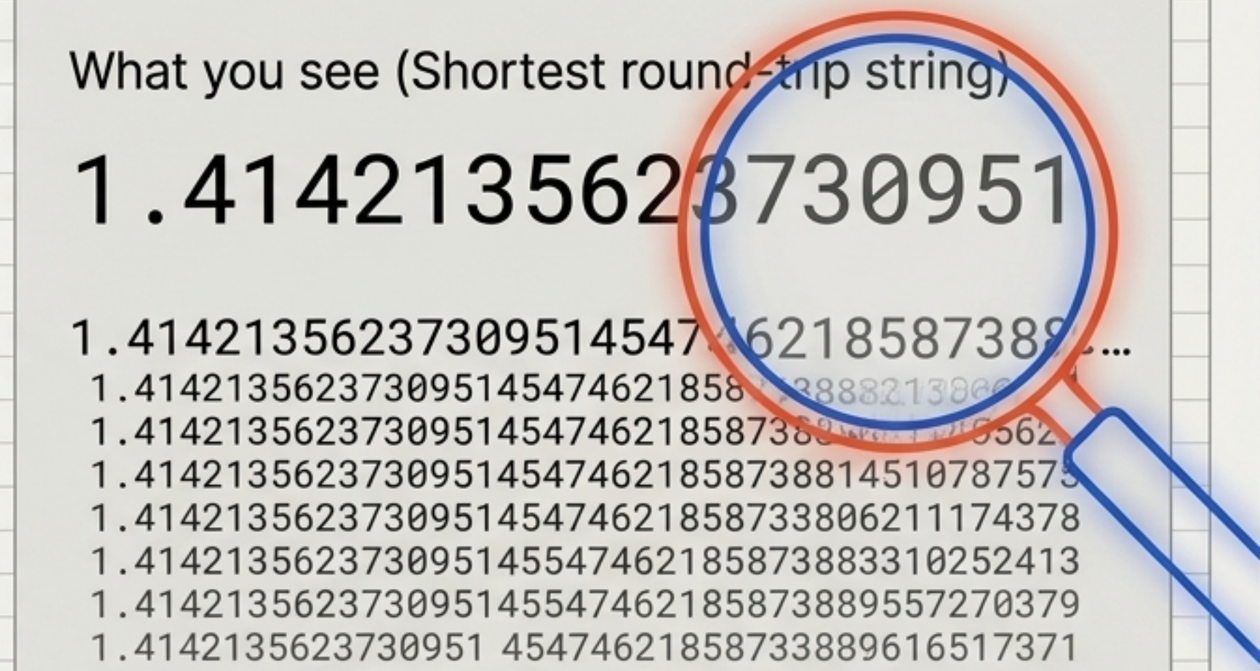
- **Rule:** Plain division (/) always produces a floating-point number, even if it divides evenly.
- **The Illusion:** Floats are stored as IEEE doubles (good for ~15-17 significant digits). They are approximations. `print()` only shows the shortest decimal string that round-trips to that exact stored double.

```
print(33/33)

1.0
```

(integer division returns float)

```
print(1/5)

0.2
```

X-Ray Magnifying Glass



What is stored (IEEE double)

Exponentiation (**)

Definition: ** represents repeated multiplication.
Rule: Non-integer exponents return floats.

Integer Exponents

```
print(2**3)
```

8

```
print(2 * 2 * 2)
```

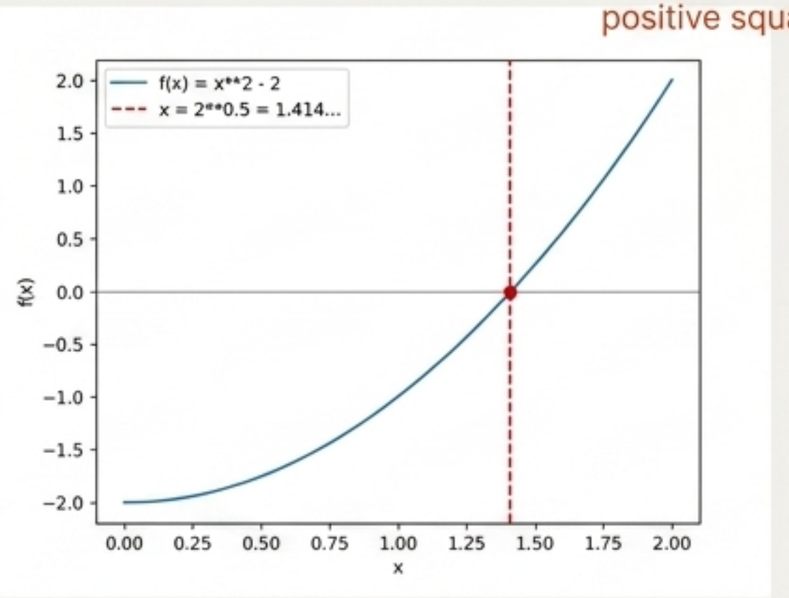
8

Non-Integer Exponents

```
print(2**0.5)
```

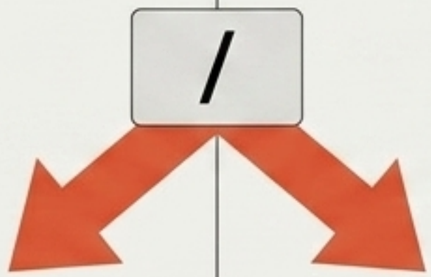
1.4142135623730951

A non-integer exponent acts as a root. This calculates the positive square root of 2.



Floor Division (//) & Modulo (%)

Precedence: // and % share the exact same precedence as * and / (evaluated left-to-right).



The Floor (//)

Returns the largest integer less than or equal to the true result.

```
print(85 // 2) -> 42
```

True result is 42.5, rounded down

The Remainder (%)

Returns the remainder of a standard division.

```
print(10 % 2) -> 0
```

Divides evenly

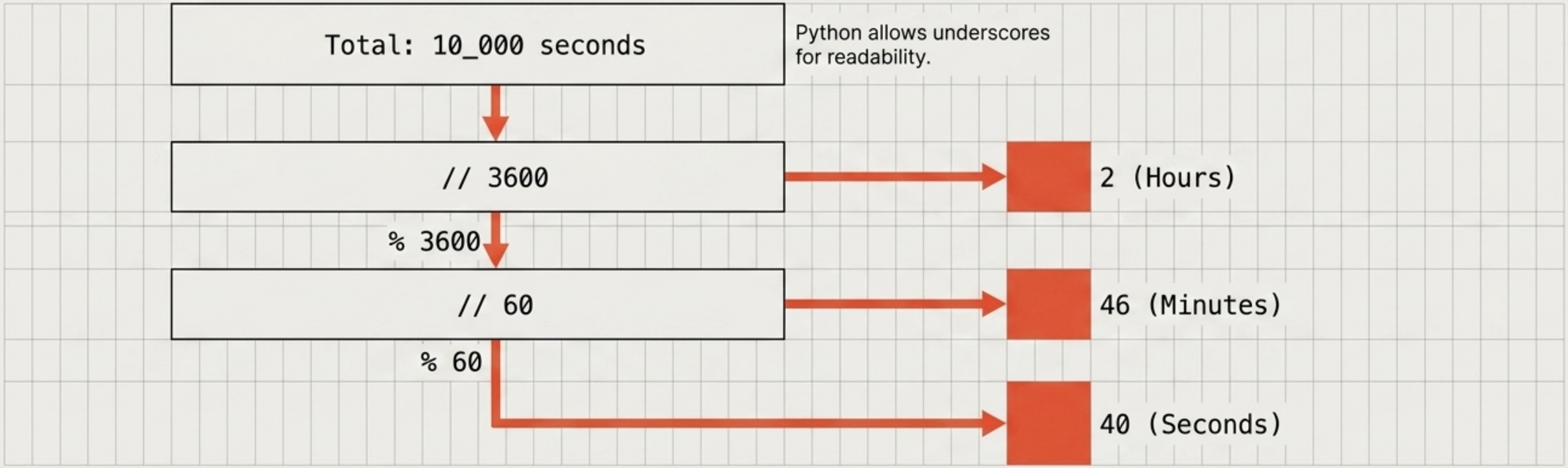
```
print(11 % 2) -> 1
```

11/2 is 5 with remainder 1

The Units-Splitting Idiom

The Idiom: Use `//` to extract the largest unit, and `%` to pass the remainder down to the smaller unit.

Worked Example 2: Seconds to H:M:S



Worked Example 1 (Minutes to H:M)

```
153 // 60 -> 2 (Hours)
153 % 60 -> 33 (Minutes)
Result: 153 minutes are 2 hours and 33 minutes.
```

round() & Banker's Rounding

Rule:
Round-half-to-even ('banker's rounding'). Exact ties round to the nearest even choice to prevent statistical drift.

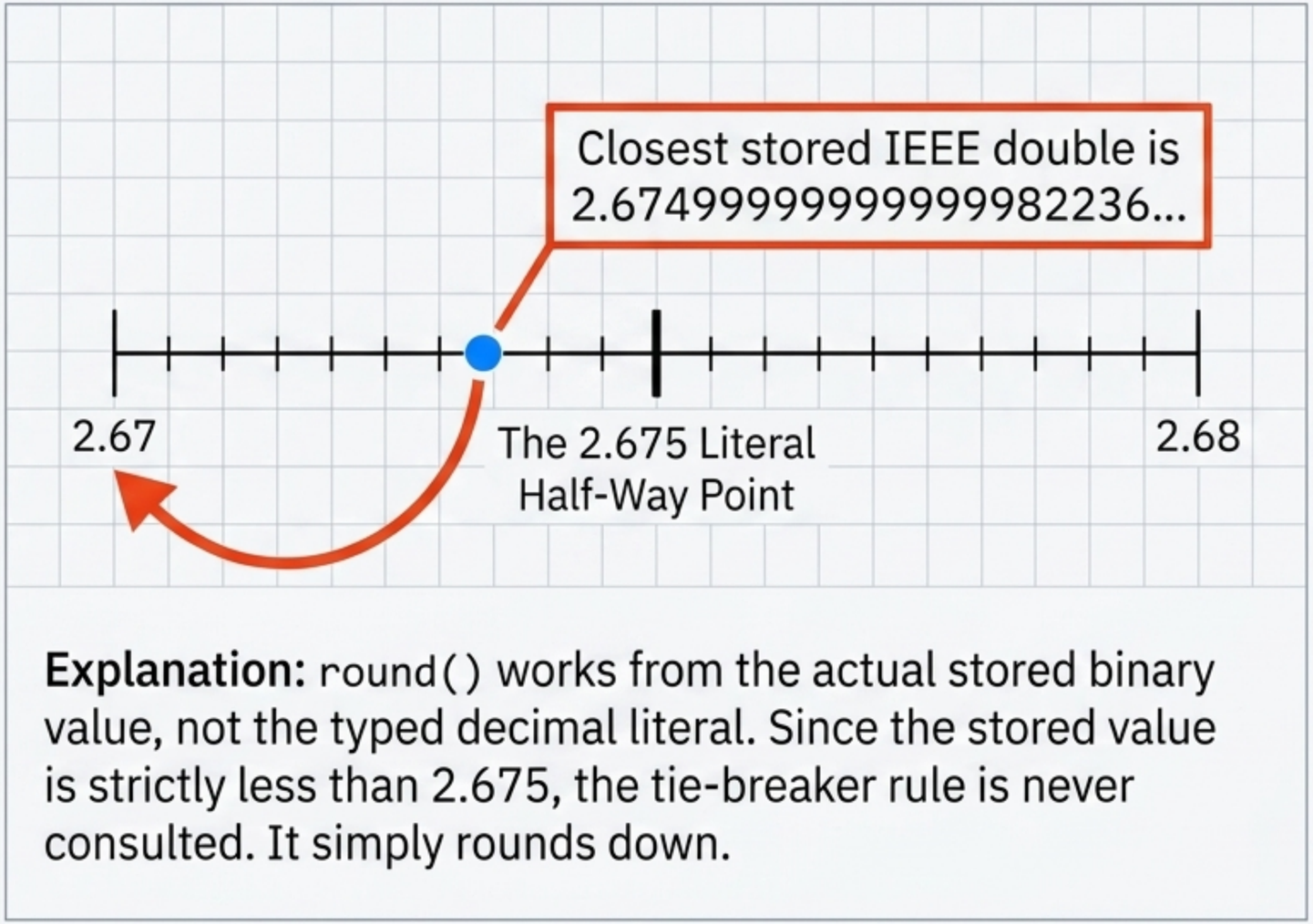
```
round(111.5) -> 112
round(3.5) -> 4
```

Warning Callout Box

The Trap

```
print(round(2.675, 2))
```

Output: `2.67` (Wait, why not 2.68?)



Explanation: `round()` works from the actual stored binary value, not the typed decimal literal. Since the stored value is strictly less than 2.675, the tie-breaker rule is never consulted. It simply rounds down.

The abs() Function & Positional-Only Parameters

Rule:
Returns the absolute value (distance from zero).

```
print(abs(5)) -> 5
print(abs(-5)) -> 5
```

Doc Parsing Block

```
abs(number, /)
```

The **/** is a **positional-only marker.** It means arguments before it means arguments before it cannot be passed by their parameter name.

Error Proof (The Consequence)

```
python3 -c "abs(x=-5)"
TypeError: abs() takes no keyword arguments
```

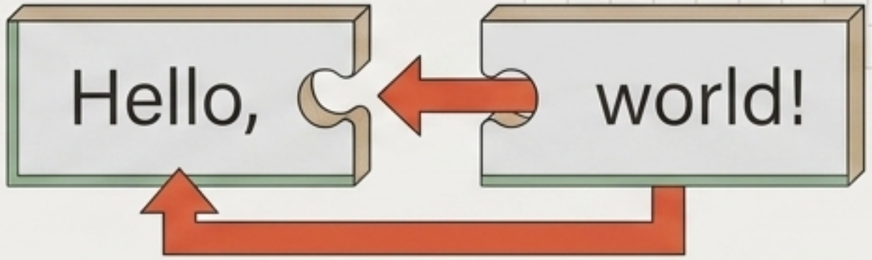
Strings: Operators & Concatenation

Rule: Strings are sequences of characters in quotes. Math operators don’t do math on strings; they do assembly.

Concatenation (+)

```
print("Hello," + " world!")
```

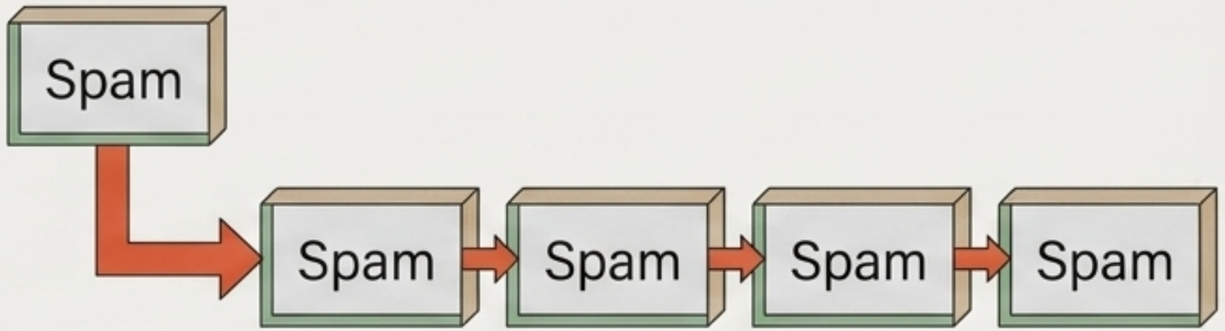
```
Hello, world!
```



Repetition (*)

```
print("Spam " * 5)
```

```
Spam Spam Spam Spam Spam
```



String Methods & Type Mixing Errors

Valid String Functions & Methods

Built-in Function (takes string as an argument):

```
print(len("hello")) -> 5
```

Dot-Operator Methods (attached directly to the object):

```
print("hello".upper()) -> HELLO
print("HELLO".lower()) -> hello
```

Dot operator calls method on object

The Type Mixing Trap (Crashes)

Attempting to implicitly mix types raises a TypeError.

Trigger 1

Multiplying the result of the print function, not the string.

```
print('Spam') * 5
```

TypeError: unsupported operand type(s) for *: 'NoneType' and 'int'

Trigger 2

Explicit conversion required: "The answer is " + str(42)

```
"The answer is " + 42
```

TypeError: can only concatenate str (not "int") to str

Values, Types & Dynamic Conversion

Rule: Python assigns memory and type automatically based on the value (Dynamic Typing).

int (Integer)

```
type(2)
```

float (Floating-point)

```
type(33.3)
```

str (String)

```
type("hello")
```

bool (Boolean)

```
type(True)
```

Type Conversion Pipeline (Casting)

```
int(33.9) -> 33
```

Truncates, does NOT round!

```
float(33) -> 33.0
```

```
str(42) -> '42'
```

Conversion Crash

```
int('hello')
ValueError: invalid literal for int() with base 10: 'hello'
```

Right type (string) but unacceptable value.

The Error Taxonomy (Diagnostic Matrix)

Error Type	Description / Trigger	Code Example
Syntax **Errors** (Caught **before running**)		
`SyntaxError`	Bad grammar / missing characters.	`print('hello')`
`IndentationError`	Bad spacing.	`  y = 2`
Runtime Errors (Crashes at the failing line)		
`NameError`	Undefined variable.	`print(age)`
`TypeError`	Wrong type for operation.	`'3' + 3`
`ValueError`	Right type, impossible value.	`int('hello')`
`ZeroDivisionError`	Dividing by zero.	`10 / 0`
`ModuleNotFoundError`	Missing import.	`import pandamonium`
`IndexError`	Out of bounds.	`x = [1,2]; print(x[5])`
Semantic Errors (The Sneakiest)		
Semantic Error	Runs perfectly, no message, wrong answer.	`area = 2 * 3.14 * radius`

Glossary: Math & Operators

`expression`

Any mix of values/operators that reduces to a single value.

`operator precedence`

The order operators apply (PEMDAS).

`floor division (//)`

Division rounded down to a whole number.

`modulo (%)`

The remainder left after a division.

`exponentiation (**)`

Repeated multiplication.

`float`

Number with a decimal point, stored as IEEE double (~15-17 digits).

`banker's rounding`

Exact ties round to the even choice (Python's round rule).

`truncation`

Cutting digits off without rounding (e.g., `int(33.9)`).

`edge case`

An input outside the usual range that breaks assumptions.

Glossary: Types, Functions & Errors

`dynamic typing`	Python assigns a value's type as the program runs, not in advance.
`argument`	The value passed to a function when it is called.
`parameter`	A name in a function's definition for an input it accepts.
`positional-only (/)`	Marker in a signature; parameters before it cannot be passed by name.
`call`	To run a function by naming it with parentheses.
`traceback`	The report Python prints when an uncaught error stops it.
`syntax error`	Code Python cannot parse; caught before anything runs.
`runtime error`	An error that stops the program while it is running.
`semantic error`	Code runs but gives the wrong answer (no error message).
`exception`	An error raised while a program runs; can be caught instead of crashing.